Atelier 1

Conception sécurisée et modélisation des menaces

ToutBet' : plateforme mobile-first de paris personnalisés entre particuliers

Membres du groupe

Nom	Compte GitHub
Maël Bourdin	mael-bourdin
(à compléter)	Airkyzzz
(à compléter)	simon-grd

Dépôt : https://github.com/mael-bourdin/toutbet-devsecops

Formation DevSecOps - EPSI 2026-2027

Sommaire

1. Backlog produit / MVP : User Stories et Abuse Stories
2. Architecture et frontières de confiance
3. Modélisation STRIDE des menaces
4. Prototype : prompts utilisés et dépôt Git

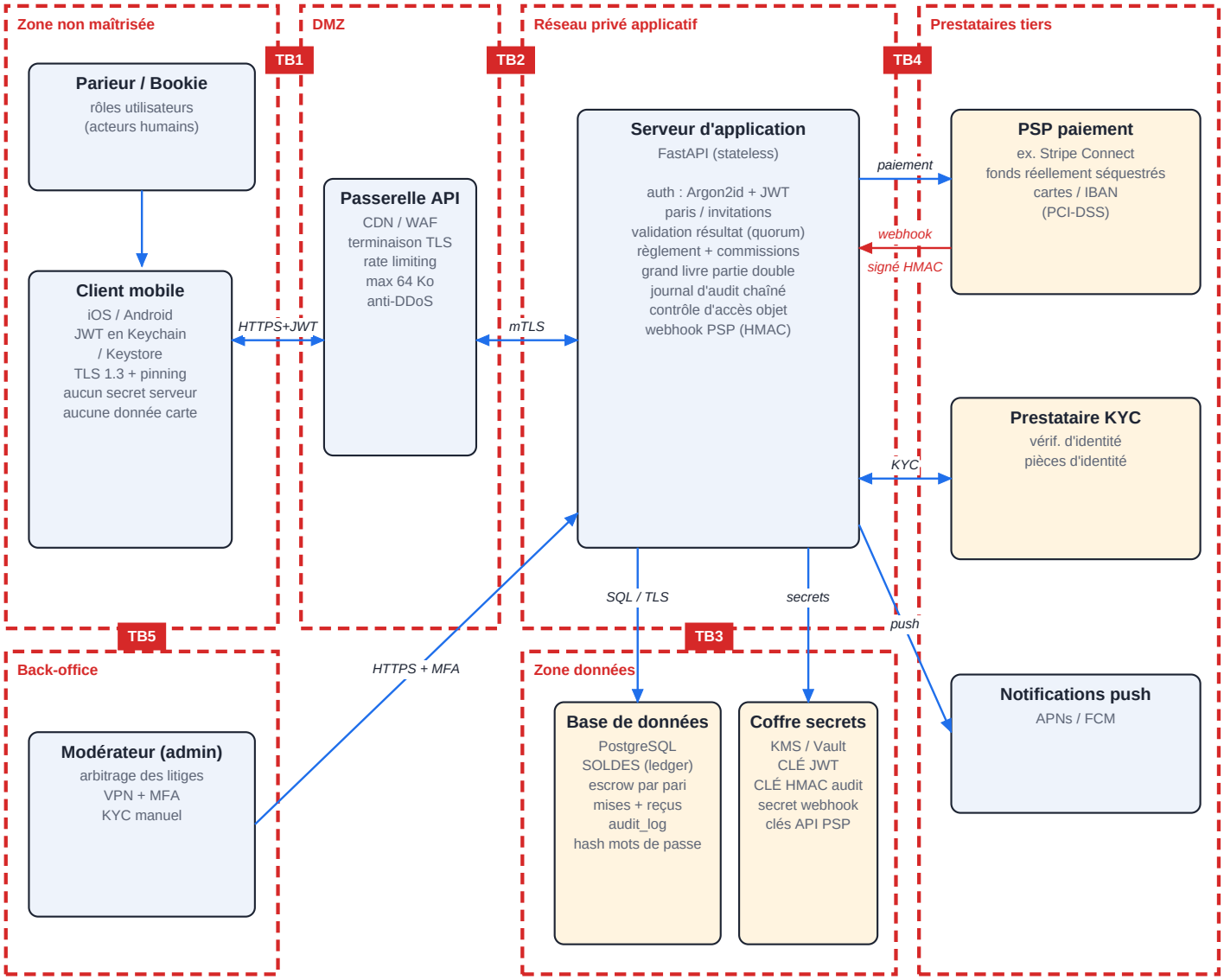
Démarche : chaque User Story du MVP est accompagnée d'au moins une Abuse Story. Les Abuse Stories alimentent la matrice STRIDE, qui a elle-même été injectée dans le prompt de génération du code. Chaque Abuse Story est enfin vérifiée par un test automatisé (`tests/test_abuse_stories.py`), exécuté en intégration continue.

1. Backlog produit / MVP

ID	User Story	Abuse Story	Mitigation / test
US-01	En tant que visiteur, je veux créer un compte (pseudo, e-mail, mot de passe) pour rejoindre la plateforme.	**AS-01** En tant qu'attaquant, je veux tester des milliers de couples identifiant/mot de passe fuités (credential stuffing) et savoir quels pseudos existent pour cibler des comptes.	Argon2id, mot de passe >= 12 car., limitation 5 essais/min, message d'erreur générique, temps constant. `test_as01_*`
US-02	En tant qu'utilisateur, je veux me connecter et rester connecté de manière sécurisée sur mon mobile.	**AS-02** En tant qu'attaquant, je veux forger ou rejouer un jeton de session (alg=none, clé devinée) pour agir au nom d'un autre parieur.	JWT HS256 figé, iss/aud/exp obligatoires, durée 15 min, secret >= 32 octets hors code (KMS). `test_as02_*`
US-03	En tant que bookie, je veux créer un pari (« Matis va-t-il réussir son créneau ? ») avec une date de clôture, une mise minimum et ma commission.	**AS-03** En tant que bookie malveillant, je veux fixer une commission de 90 % ou injecter des champs cachés (platform_fee_bps, role) pour détourner les gains.	Schémas stricts (extra=forbid), commission plafonnée à 10 % (validation + contrainte SQL), frais plateforme fixés serveur. `test_as03_*`
US-04	En tant que bookie, je veux inviter des parieurs par leur pseudo pour que seuls mes amis puissent parier.	**AS-04** En tant qu'utilisateur non invité, je veux deviner l'identifiant d'un pari privé pour le consulter et y miser (IDOR).	Contrôle d'accès au niveau objet (bookie / invité / admin), réponse 404 indiscernable, UUID v4. `test_as04_*`
US-05	En tant que parieur, je veux miser 10 € sur « oui » ou « non » pour montrer que j'ai raison.	**AS-05** En tant que parieur malveillant, je veux modifier mon pari après sa clôture pour remporter la mise. **AS-06** En tant que parieur malveillant, je veux rejouer ma requête de mise (double clic, script) pour provoquer une double dépense ou une incohérence de solde. **AS-07** En tant que parieur malveillant, je veux miser un montant négatif ou supérieur à mon solde.	Mise immuable (aucune route PUT/PATCH/DELETE), clôture à l'heure serveur, 1 mise/parieur (contrainte unique), Idempotency-Key, montants entiers > 0, vérification de solde dans la transaction. `test_as05_*, test_as06_*, test_as07_*`
US-06	En tant que parieur, je veux approvisionner mon portefeuille par carte bancaire.	**AS-10** En tant qu'attaquant, je veux envoyer un faux webhook « paiement réussi » (ou rejouer un vrai, ou en changer le montant) pour créditer mon compte sans payer.	Crédit uniquement via webhook PSP signé HMAC + horodatage (5 min), anti-rejeu par event_id, montant comparé à l'intention de paiement ; aucune donnée carte stockée (PCI-DSS délégué au PSP). `test_as10_*`
US-07	En tant que bookie, je veux déclarer le résultat du pari une fois l'événement terminé.	**AS-08** En tant que bookie malveillant, je veux parier sur mon propre pari puis déclarer le résultat qui m'arrange. **AS-09a** En tant que bookie malveillant, je veux déclarer un faux résultat pour favoriser un complice, ou le déclarer avant la clôture.	Le bookie ne peut pas miser ; résultat proposable seulement après clôture ; le résultat n'est qu'une proposition soumise à validation des parieurs. `test_as08_*, test_as09_*`
US-08	En tant que parieur, je veux confirmer ou contester le résultat proposé avant que l'argent ne soit redistribué.	**AS-09b** En tant que parieur malveillant, je veux voter plusieurs fois ou créer de faux comptes (Sybil) pour faire valider un faux résultat.	Un vote par parieur (contrainte unique), seuls les parieurs votent, quorum > 50 % ; >= 1/3 de contestations => litige arbitré par un modérateur ; invitation nominative + KYC limitent les comptes multiples. `test_as09_*`
US-09	En tant que gagnant, je veux recevoir automatiquement ma part, commissions bookie et plateforme déduites.	**AS-14** En tant qu'utilisateur malveillant, je veux exploiter les arrondis ou une erreur de calcul pour créer de l'argent ou siphonner des centimes.	Montants en centimes entiers, grand livre en partie double (somme = 0 vérifiée), reliquat d'arrondi déterministe, règlement atomique et idempotent. `test_invariant_*, test_compute_payouts_*`
US-10	En tant qu'utilisateur, je veux retirer mes gains vers mon compte bancaire.	**AS-11** En tant qu'attaquant ayant volé un jeton (ou une identité), je veux vider le portefeuille de la victime.	Ré-authentification par mot de passe, KYC obligatoire avant retrait, journalisation, (prévu : MFA, délai sur nouvel IBAN). `test_as11_*`
US-11	En tant que parieur, je veux consulter l'historique de mes mises avec un reçu pour prouver ce que j'ai parié.	**AS-12** En tant que perdant malveillant, je veux nier avoir effectué le pari ; en tant qu'initié, je veux effacer ou modifier une trace dans les logs.	Reçu HMAC de chaque mise, journal d'audit chaîné par HMAC (toute altération détectée), export WORM prévu. `test_as12_*`
US-12	En tant que parieur, je veux pouvoir parier même pendant la finale de la Coupe du monde.	**AS-13** En tant qu'attaquant, je veux saturer l'API (flood, requêtes énormes) lors d'un événement majeur pour empêcher les paris concurrents.	Limitation de débit par utilisateur/IP, taille de requête bornée (64 Ko), WAF/CDN + autoscaling en production. `test_as13_*`

2. Architecture technique et frontières de confiance

Les cadres **rouges en pointillés** matérialisent les zones de confiance : tout flux qui traverse une ligne rouge franchit une frontière (TB1 à TB5) et doit être authentifié, chiffré et validé. Les composants en orange contiennent des données sensibles.



Frontières de confiance

ID	Changement de confiance
TB1	Smartphone de l'utilisateur <-> passerelle API (Internet, appareil non maîtrisé)
TB2	Passerelle/WAF <-> serveur d'application (réseau privé)
TB3	Serveur d'application <-> zone données (BDD, coffre de secrets)
TB4	Plateforme <-> prestataires tiers (PSP de paiement, KYC, notifications push)
TB5	Back-office modérateur <-> API (accès privilégié, VPN + MFA)

Flux de données principaux

#	Sens	Échange	Frontière
1	Mobile -> API	Inscription / connexion ; l'API renvoie un JWT de 15 min.	TB1
2	Mobile -> API	Création de pari, invitations, mise (Idempotency-Key) ; reçu HMAC renvoyé.	TB1, TB2
3	API -> BDD	Débit du compte parieur, crédit du compte escrow du pari (transaction équilibrée).	TB3
4	API -> PSP -> API	Intention de dépôt, paiement chez le PSP, puis webhook signé qui crédite le solde.	TB4
5	Mobile -> API	Bookie : proposition du résultat après clôture ; parieurs : confirmation / contestation.	TB1
6	API -> BDD	Règlement : escrow -> gagnants, commission bookie, commission plateforme.	TB3
7	Admin -> API	Arbitrage d'un litige, validation KYC.	TB5
8	API -> PSP / KYC	Retrait (ré-authentification + KYC), vérification d'identité.	TB4

Localisation des données sensibles

Donnée sensible	Emplacement / protection
Secrets (clé JWT, clé HMAC d'audit, secret webhook PSP, clés API PSP/KYC)	Gestionnaire de secrets / KMS, injectés en variables d'environnement ; jamais dans le code ni l'app mobile
Soldes utilisateurs et fonds séquestrés	BDD : grand livre ledger_entries (comptes user:*, escrow:*, platform, psp) ; les fonds réels sont chez le PSP
Mises, reçus, votes	BDD : tables wagers, result_votes
Mots de passe	BDD : hash Argon2id uniquement
Journal d'audit	BDD audit_log chaîné + export WORM (prévu)
Données d'identité (KYC), IBAN, carte	Chez les prestataires KYC/PSP ; la plateforme ne garde qu'un statut / jeton
Jeton de session	Smartphone : Keychain (iOS) / Keystore (Android)

3. Modélisation STRIDE des menaces

Catégorie	Menace identifiée	Composant visé	Mitigation proposée	Statut MVP
Spoofing	Credential stuffing / brute-force sur un compte	API /auth	Argon2id, rate limit 5/min par IP+pseudo, message générique, MFA (prévu)	Implémenté (MFA prévu)
Spoofing	Jeton JWT forgé (alg=none, clé faible) ou volé	API, client mobile	Algorithme figé, iss/aud/exp requis, TTL 15 min, secret en KMS, stockage Keychain/Keystore	Implémenté
Spoofing	Faux webhook PSP « paiement réussi »	Endpoint /payments/webhook	Signature HMAC + horodatage, comparaison temps constant, allowlist IP du PSP	Implémenté (allowlist prévue)
Spoofing	Fraude à l'identité / comptes multiples (Sybil)	Comptes, vote	KYC via prestataire avant retrait, invitations nominatives, détection appareil (prévu)	Partiel
Tampering	Modifier une mise après la clôture	API paris, BDD	Mise immuable, heure serveur, contrainte unique, aucune route de modification	Implémenté
Tampering	Montant manipulé côté client (négatif, > solde, montant PSP différent)	API, PSP	Validation stricte, contrôle de solde transactionnel, montant webhook = intention	Implémenté
Tampering	Double dépense par rejeu / condition de concurrence	Grand livre	Idempotency-Key, transaction ACID, verrou optimiste ; SERIALIZABLE/SELECT FOR UPDATE sur PostgreSQL	Implémenté
Tampering	Altération directe des soldes en base	BDD	Grand livre append-only en partie double, solde calculé, compte SQL sans UPDATE/DELETE	Implémenté (droits SQL prévus)
Tampering	Interception/modification mobile <-> API (MITM)	Réseau	TLS 1.3, HSTS, certificate pinning dans l'app	HSTS impl., pinning prévu
Repudiation	Un perdant nie avoir effectué le pari	Logs / BDD	Reçu HMAC de la mise (signature électronique) + journal non répudiable chaîné	Implémenté
Repudiation	Le bookie nie avoir déclaré un résultat	Journal d'audit	Entrée horodatée avec actor_id dans la même transaction que l'action	Implémenté
Repudiation	Un administrateur efface/modifie une trace	Journal d'audit	Chaîne HMAC (vérification), export vers stockage WORM externe	Chaîne impl., WORM prévu
Information disclosure	IDOR : lecture d'un pari privé et de ses mises	API paris	Contrôle d'accès objet, 404 indiscernable, UUID non séquentiels	Implémenté
Information disclosure	Fuite de secrets (clés JWT/HMAC/PSP) dans le dépôt	Dépôt, CI	Variables d'environnement, refus de démarrer sans secret, gitleaks en CI, gestionnaire de secrets	Implémenté
Information disclosure	Fuite de détails techniques ou de données bancaires	API, BDD	Erreurs génériques sans stacktrace, aucune donnée carte (PSP), en-têtes no-store	Implémenté
Denial of service	Saturation lors d'un événement majeur	API, infra	Rate limiting, CDN/WAF, autoscaling, file d'attente pour le règlement	Rate limit impl., infra prévue
Denial of service	Requêtes géantes / épuisement CPU (hash Argon2)	API	Corps limité à 64 Ko, rate limit appliqué avant le hachage	Implémenté
Elevation of privilege	Mass assignment pour devenir admin ou changer les frais	API	Schémas extra=forbid, rôle relu en base, frais plateforme fixés serveur	Implémenté
Elevation of privilege	Un parieur déclare le résultat / le bookie parie sur son pari	Logique métier	Contrôle par rôle sur l'objet (bookie, parieur, modérateur)	Implémenté
Elevation of privilege	Le bookie impose un faux résultat pour débloquer l'escrow	Validation résultat	Quorum des parieurs, litige si >= 1/3 conteste, arbitrage modérateur uniquement	Implémenté

Risques résiduels

- Rate limiter en mémoire : à remplacer par Redis derrière plusieurs instances.

- Collusion d'une majorité de parieurs avec le bookie : atténuée par le seuil de contestation (1/3) et la modération, non éliminée ; prévoir preuves (photo/vidéo) et réputation.

- Pas de MFA ni de certificate pinning dans le MVP (application mobile non livrée).

- Intégrations PSP/KYC simulées : à brancher sur un prestataire agréé (ex. Stripe Connect, Onfido).

- Aspects réglementaires (ANJ, LCB-FT) hors du périmètre technique de l'atelier.

4. Prototype en vibe coding

Modèle utilisé : **Claude Opus 5.5** (Anthropic) dans Claude Code, retenu pour la qualité du code produit et sa capacité à exécuter puis corriger les tests. Le prompt reprend les recommandations du cours (secure by design, moindre privilège, aucun secret en dur, tests des abuse stories, pipeline DevSecOps) et embarque explicitement le backlog et la matrice STRIDE.

Dépôt public : https://github.com/mael-bourdin/toutbet-devsecops

P1 - Génération du socle (Claude Opus 5.5 via Claude Code)

```
Rôle : tu es un développeur backend senior spécialisé en sécurité applicative (DevSecOps).

Contexte : nous lançons « ToutBet' », une plateforme mobile-first de paris personnalisés entre particuliers. Un
Bookie crée un pari (ex. « Matis va-t-il réussir son créneau ? »), fixe éventuellement une mise minimum et sa
commission, invite des Parieurs. L'argent est séquestré par la plateforme puis redistribué aux gagnants après
validation du résultat, déduction faite des commissions bookie et plateforme.

Objectif : générer le socle technique de l'API backend du MVP en Python 3 / FastAPI / SQLAlchemy 2 (SQLite en dev,
compatible PostgreSQL), avec une suite de tests pytest.

Périmètre fonctionnel (User Stories) : [coller le backlog US-01 à US-12]

Exigences de sécurité (Secure by Design, OWASP ASVS niveau 2, OWASP API Top 10) :
1. Chaque Abuse Story du backlog [coller AS-01 à AS-14] DOIT être bloquée et couverte par au moins un test automatisé
nommé test_asXX_*.
2. Chaque mitigation de la matrice STRIDE [coller le tableau] marquée « Implémenté » doit apparaître dans le code.
3. Aucun secret en dur : configuration par variables d'environnement, refus de démarrer si un secret manque ou fait
moins de 32 caractères ; fournir un .env.example sans valeur réelle.
4. Authentification : Argon2id, JWT à algorithme figé avec iss/aud/exp, durée courte ; messages d'erreur génériques ;
limitation de débit sur la connexion.
5. Autorisation au niveau objet sur chaque route (anti-IDOR), principe du moindre privilège, rôle relu en base.
6. Validation stricte des entrées (pydantic, extra=forbid), montants en centimes entiers, jamais de float.
7. Argent : grand livre en partie double append-only, transactions atomiques, clés d'idempotence, invariant « somme
des écritures = 0 ».
8. Paiement : le solde n'est crédité que par un webhook PSP signé HMAC, horodaté, anti-rejeu.
9. Non-répudiation : reçu signé HMAC pour chaque mise, journal d'audit chaîné et vérifiable.
10. Disponibilité : limitation de débit des écritures, taille de requête bornée.
11. En-têtes de sécurité HTTP, pas de stacktrace exposée.
12. Dépendances épinglées ; pipeline CI GitHub Actions : tests, SAST (bandit), audit des dépendances (pip-audit),
détection de secrets (gitleaks) ; Dependabot.

Contraintes de qualité : code lisible, commenté là où la sécurité l'exige, sans fonctionnalité hors périmètre ;
explique les risques résiduels que le MVP ne couvre pas.
Livrables : arborescence complète, README (installation, lancement, tests), et lance les tests jusqu'à ce qu'ils
passent tous.
```

P2 - Revue de sécurité du code généré

```
Relis le code généré avec un regard d'attaquant : pour chaque ligne de la matrice STRIDE, indique le fichier et la
fonction qui implémentent la mitigation, ou signale qu'elle manque. Propose un test qui échouerait si la mitigation
était retirée.
```

P3 - Correction et tests

```
Exécute la suite pytest, corrige les échecs sans affaiblir les contrôles de sécurité, puis vérifie que bandit et pip-
audit ne remontent aucune alerte bloquante.
```

Résultat des tests

Socle généré : API FastAPI (auth, paris, invitations, mises, validation du résultat par quorum, litiges, règlement, portefeuille, webhook PSP), grand livre en partie double et journal d'audit chaîné. **24 tests pytest passent** (3 parcours nominaux, 21 tests d'abuse stories et d'invariants). La CI GitHub Actions exécute pytest, bandit (SAST), pip-audit (dépendances) et gitleaks (secrets) à chaque push.